联邦微调综合项目任务书

云计算与人工智能前沿 • 2026 年秋

一、项目概述

银行客服每天收到大量提问，需要自动判断用户意图（如“卡片何时寄到”“转账失败”）。现实中，这些对话分散在不同机构，出于隐私不能集中训练。本项目围绕这一任务完成三个任务：大模型适配与推理、分布式训练与联邦学习、真实约束下的联邦挑战。

项目	统一规定
基座模型	Qwen2.5-1.5B-Instruct
数据集	Banking77：训练集 9993 条，77 类；公开测试集 3076 条
数据划分	课程发布的 split.json：验证集 1001 条；其余样本分给 10 个客户端，提供 iid、dir1.0、dir0.1 三种划分
测试	任务一、二使用公开测试集；任务三由课程组使用未公开的测试集统一评测
计算平台	交大超算 π 2.0 dgx2 队列（V100 32GB）；排队时可用思源一号 debuga100 与 CPU 队列。见《超算使用手册》《GPU 排队说明与替代方案》
评价指标	准确率（accuracy），同时报告 macro-F1

dir 划分按 Dirichlet 分布生成，α 越小客户端之间差异越大。iid 下每个客户端都有全部 77 类；dir0.1 下多数客户端只有 22–39 类。验证集用于选择方法与参数，测试集只用于报告结果。

二、课程提供的材料

文件	用途
split.json	统一数据划分，请勿修改或重新划分
scenario.json	任务三场景约束，请按此文件完成任务三
evaluate.py	统一评测脚本
setup_env.sh	建立 Python 环境
download.sbatch	通过镜像下载模型与数据
job.sbatch / job_dist.sbatch	单进程 / 单卡多进程 GPU 作业模板
setup_env_sy.sh、download_sy.sbatch、job_debug*.sbatch、job_cpu*.sbatch	思源一号的环境配置与作业模板（debuga100、CPU 队列）
超算使用手册	登录、作业提交与常见问题
GPU 排队说明与替代方案	dgx2 排队时可用的平台、实测速度与用法

全部代码由学生自行编写，可以使用任何参考资料和 AI 工具。学生需对提交的全部代码和结果负责，并能在汇报中解释。三个任务可以分别编写代码。

三、任务一：大模型适配与推理（20 分）

比较“不训练”和“训练”两类方法的效果与代价，并测量模型部署时的推理性能。

1.1 零样本提示

目标：不做任何训练，仅通过提示让模型完成分类。

步骤：

1. 以对话方式加载基座模型（AutoModelForCausalLM），使用模型自带的对话模板。
2. 在系统提示中列出全部 77 个类别名，要求模型只输出一个类别名。
3. 对验证集中的每条提问生成回答（贪心解码，最多生成十几个 token）。
4. 将回答映射到类别：与类别名完全一致则直接采用，否则取最接近的类别名。
5. 计算验证集准确率。可以使用全部验证集，也可以抽样，报告中需说明。

需要记录：准确率、提示长度（token 数）、每条平均耗时、显存峰值。

需要思考：哪些类别容易出错？模型回答不是合法类别名时如何处理？

课程组参考结果：验证集抽样 300 条，准确率约 43%；提示约 400 token；每条约 0.08 秒。

提示示例：

```
[system] Classify the banking customer query into exactly one intent.
         Answer with the intent name only.
         Intents: activate_my_card, age_limit, ... (共 77 个)
[user]   Query: How do I locate my card?
         Intent:
[model]  card_arrival
```

1.2 少样本提示

目标：在提示中加入带标注的示例，观察示例数量对效果和开销的影响。

步骤：

1. 在 1.1 的提示中加入“提问 → 类别”形式的示例。
2. 示例只能从客户端训练数据中选取，不得使用验证集或测试集。
3. 至少比较两种示例数量，例如每类 1 例和每类 3 例。
4. 可以尝试不同的选例方式，如随机选取、按与当前提问的相似度检索。

需要记录：不同示例数量下的准确率、提示长度、每条平均耗时、显存峰值。

需要思考：示例越多越好吗？代价是什么？

课程组参考结果：每类 1 例约 44%（提示约 2100 token，每条约 0.15 秒）；每类 3 例约 59%（提示约 5500 token，每条约 0.47 秒）。

1.3 LoRA 集中微调

目标：用全部训练数据对模型做参数高效微调，作为后续联邦实验的参照上限。

步骤：

1. 将基座模型作为分类模型加载（AutoModelForSequenceClassification，num_labels=77）。
2. 加入 LoRA（例如秩 r=8，作用于 q_proj 和 v_proj），分类头一同训练。
3. 合并 10 个客户端的全部样本进行训练，训练若干个 epoch。
4. 每个 epoch 结束后在验证集上评估，依据验证集选择设置。
5. 报告公开测试集结果，保存适配器，并用 evaluate.py 检查能否正常加载。

需要记录：可训练参数量、显存峰值、每个 epoch 的训练时间、验证集与测试集准确率。

课程组参考结果：r=8、学习率 2e-4、批量 16、3 个 epoch：验证集约 89%，测试集约 91%；可训练参数约 120 万；显存约 10.7GB；约 4 分钟。

LoRA 参考资料：原论文 https://arxiv.org/abs/2106.09685 ；Hugging Face PEFT 文档 https://huggingface.co/docs/peft 。建议先用 SmolLM2-135M 小模型和少量样本调通代码，再换正式模型。

1.4 推理性能测量

目标：测量微调后模型在不同批量下的推理吞吐与延迟。

步骤：

1. 加载 1.3 的适配器，以 fp16 进行推理。
2. 取一批测试提问，分别在批量 1、8、32、128 下测量。
3. 正式计时前先预热一次；计时前后调用 torch.cuda.synchronize()。
4. 计算每秒处理的样本数和每批耗时，记录显存峰值。
5. 与 1.1、1.2 的每条耗时进行对比。

需要记录：各批量下的吞吐（样本/秒）、每批耗时、显存峰值。

需要思考：批量越大越快吗？在线服务与离线批处理应如何选择批量？

课程组参考结果：批量 1、8、32、128 的吞吐分别约为 32、235、540、513 样本/秒，每批耗时约 31、34、59、250 毫秒。批量增至 32 后吞吐不再提高，延迟却明显增加。

任务一提交

① 报告 • 任务一章节（最重要）。报告中的任务一章节是本任务最主要的提交物，建议包含：

- 每个子任务的做法：提示如何编写、示例如何选取、LoRA 与训练设置、推理如何计时。
- 1.1–1.3 的对比表：方法、准确率、提示长度或可训练参数量、耗时、显存。
- 1.4 的吞吐与延迟随批量变化的图。
- 分析：不训练与训练分别能做到什么程度，代价分别是什么；推理性能的规律。

② **支撑材料：**代码、运行输出、1.3 的适配器。

评定：四个子任务全部完成；报告写清做法、结果与分析，数据与日志一致；结果合理。

四、任务二：分布式训练与联邦学习（30 分）

使用 torchrun 在一张 GPU 上启动 4 个进程，进程间通过 torch.distributed（gloo 后端）通信，比较“每一步都同步”的数据并行与“每一轮才同步”的联邦平均。作业通过 job_dist.sbatch 提交，例如：sbatch job_dist.sbatch 4 my_dist.py。4 个进程共用一张卡，基座模型以 fp16 加载时每个进程约 5.3GB 显存。dgx2 排队时，也可以在思源一号 CPU 队列上用 job_cpu_dist.sbatch 运行，见《GPU 排队说明与替代方案》。

	数据并行	**联邦平均**
数据	由训练方统一切分，各进程分布相同	分布在各客户端，不能移动，分布各不相同
同步内容	梯度	模型参数（LoRA 参数与分类头）
同步频率	每一步	每一轮（本地训练若干步之后）
参与者	全部进程每步都参与	每轮只选部分客户端

2.1 多进程数据并行

目标：实现标准的分布式数据并行训练，统计其通信开销。

步骤：

1. 启动 4 个进程，合并全部客户端数据后均分，每个进程拿到 1/4。
2. 用 DistributedDataParallel 包装 LoRA 模型，反向传播时自动对梯度做 all-reduce。
3. 训练 2 个 epoch，由进程 0 在验证集上评估。
4. 统计通信量：每步同步的梯度大小 × 步数，并说明统计口径。

需要记录：每个 epoch 的验证集准确率、测试集准确率、累计通信量、总耗时、每个进程的显存。

需要思考：与 1.3 的单进程训练相比，结果是否一致？为什么？

课程组参考结果：每进程批量 16，2 个 epoch：测试集约 88%；每个 epoch 140 步；累计通信约 2580MB；约 3 分钟。

2.2 多进程联邦平均

目标：在多进程框架下实现 FedAvg，比较不同数据分布下的效果。

步骤：

1. 10 个客户端由 4 个进程分担。所有进程使用相同的随机种子，保证每轮选出相同的 4 个客户端。
2. 每个客户端从本轮全局参数出发，用自己的数据本地训练 1 个 epoch。

3. 轮末各进程对“样本数 × 参数”做 all-reduce，再除以总样本数，得到新的全局参数。
4. 在 iid 和 dir0.1 两种划分下各运行 20 轮，每 5 轮在验证集上评估一次。
5. 用一个小例子自查聚合结果是否与手工计算一致。

需要记录：每轮选中的客户端、验证集准确率、累计通信量；最终测试集准确率；总耗时。

课程组参考结果：测试集 iid 约 92%，dir0.1 约 75%；20 轮累计通信约 740MB；约 8 分钟。dir0.1 第 5 轮验证集仅约 40%，收敛明显更慢。

2.3 本地独立训练与对照分析

目标：了解“各方不合作”时的效果，并汇总对照任务一、二的结果。

步骤：

1. 在 dir0.1 划分下，每个客户端只用自己的数据训练 3 个 epoch，不与其他客户端交换任何信息。
2. 10 个模型分别在同一验证集上评估，报告平均值与范围。
3. 汇总集中微调、数据并行、联邦平均（iid、dir0.1）、本地独立训练的结果与通信量，形成对照表。

需要记录：每个客户端的验证集准确率；对照表。

课程组参考结果：本地独立训练的验证集准确率平均约 27%（各客户端 19%–34%）。

任务二提交

① 报告·任务二章节（最重要）。报告中的任务二章节是本任务最主要的提交物，建议包含：

- 代码结构：数据并行与联邦平均分别在哪里同步、同步什么。
- 聚合自查的结果；通信量是怎么统计的。
- 对照表与 2.2 的准确率–轮次曲线。
- 分析：数据并行与联邦平均在通信量和效果上的取舍；数据越不均衡，联邦平均为什么越差；本地独立训练说明了什么。

② 支撑材料：代码、作业脚本、运行输出。

评定：三个子任务全部完成；聚合正确，通信量统计合理；报告写清做法、结果与分析，数据与日志一致；结果在合理范围内。

五、任务三：真实约束下的联邦挑战（20 分）

在 scenario.json 规定的场景下训练一个联邦全局模型，在满足全部约束的前提下使准确率尽可能高。场景同时包含 Lecture 3 介绍的通信代价、系统异构和统计异构三类挑战。方法不限，由学生自行查阅资料、设计和验证。

约束	规定
数据	dir0.1 划分；不得汇集客户端原始数据；验证集

	与公开测试集不得用于训练；不得使用课程数据以外的数据或其他模型
通信	全程上传总量不超过 115 MB，按实际上传的字节数计算，下发不计入
轮次与参与	最多 40 轮，每轮最多 5 个客户端；每个客户端每轮最多遍历本地数据 1 次
弱设备	客户端 0、4、7 每轮最多处理 160 个样本（批量 16 时即 10 步）
掉线	drop_table[r] 列出第 r 轮（从 0 开始）掉线的客户端，被选中也不能上传；选择客户端时不得利用当前及之后轮次的掉线信息
选择依据	方法与参数只能依据验证集选择

scenario.json 的结构如下：

```
{
  "partition": "dir0.1",
  "upload_budget_mb": 115,
  "max_rounds": 40,
  "max_clients_per_round": 5,
  "max_local_epochs": 1,
  "weak_clients": [0, 4, 7],
  "weak_max_samples_per_round": 160,
  "drop_table": [[0, 4], [6], [9], [7], [4], ...]
}
```

程序读取 scenario.json，全班面对完全相同的场景。请在日志中记录每轮参与的客户端和累计上传量，并在报告中给出最终的累计上传量。如对上传内容进行压缩，按压缩后实际发送的字节数计算，服务器端应能据此还原参数。

参考起点：直接使用任务二的联邦平均（上传不压缩），预算只够约 6 轮，验证集准确率约 46%。

课程组使用未公开的测试集，以 evaluate.py（截断长度 64）统一评测提交的最终适配器。该测试集由课程组另行编写，提问风格可能与公开数据略有不同，请关注模型的泛化能力。任务三成绩依据最终测试指标，并结合方法与分析评定；不符合场景设定的结果不予认可。改进没有效果时，如实报告并分析原因，同样有价值。

任务三提交

① 报告·任务三章节（最重要）。报告中的任务三章节是本任务最主要的提交物，建议包含：

- 方法说明：针对哪些约束、原理是什么、参考了哪些资料。
- 验证集上的对照实验：与参考起点相比，每项改动带来的变化；如有条件，报告多个随机种子的结果。
- 累计上传量等场景设定的遵守情况。
- 分析：有效与无效的原因，局限与改进方向。

② **支撑材料：**最终全局适配器（final_adapter/，须能用 evaluate.py 直接加载）、代码、运行日志。

六、提交

第 13 教学周最后一天前，通过 Canvas 一次提交：

1. 实验报告（PDF，最重要的提交物），按三个任务分章节，每章包含做法、结果表与曲线、分析（具体见各任务的“提交”部分）；
2. 全部代码与作业脚本；
3. 运行输出与日志；
4. 任务一 1.3 的适配器与任务三的最终适配器（final_adapter/ 目录，须能用 evaluate.py 直接加载）；
5. 汇报幻灯片。

七、评分

组成	**比例**
考勤	10%
项目材料（任务一 20 分、任务二 30 分、任务三 20 分）	70%
项目汇报	20%

任务一、二按完成情况与正确性评定；任务三依据最终测试指标，并结合方法与分析评定。教师和助教审核提交材料，并结合课堂汇报中的讲解与问答综合评定。

八、课堂汇报

每位同学先陈述、后问答，具体时长与顺序另行通知。汇报幻灯片随报告一起提交，内容与形式自定。

评分关注：结果与分析是否清楚；任务三的方法与效果是否讲清；关键代码讲解：挑一两段本人认为最关键的代码，讲清做了什么、为什么这样写。有价值的发现、失败的排查过程等亮点会在评定中体现。

提问方面：问题围绕每位同学本人的结果和做法提出，主要涉及结果中的具体数字、参数设置、任务三的方法以及本人的发现。

九、算力与注意事项

- 每人同时只能运行 1 个作业、使用 1 张 GPU；单个作业最长 12 小时。
- 登录节点禁止运行训练，必须通过 sbatch 或 srun 在计算节点运行。

- 全部实验约需 3–5 个 GPU 小时。GPU 全校共享，临近截止时排队会明显变长，请尽早开始。dgx2 排队时，可改用思源一号的 debuga100 和 CPU 队列，见《GPU 排队说明与替代方案》。
- 先用 SmolLM2-135M 小模型和少量样本调通代码，再换正式模型。
- 遇到问题在课程群里提问，或联系助教（发消息、发邮件均可），附上作业号和 .err 文件最后 20 行。